ACADFLOW: DIGITAL ACADEMIC DOCUMENT WORKFLOW & MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

MOHAMMED RAYYAN L (310621104055)
KISHORE KUMAR S (310621104043)
LOKESHWARAN P (310621104048)

in partial fulfillment for the award of the Degree
of
BACHELOR OF ENGINEERING
in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



EASWARI ENGINEERING COLLEGE,
(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY,
CHENNAI

APRIL 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"ACADFLOW: DIGITAL ACADEMIC DOCUMENT WORKFLOW & MANAGEMENT SYSTEM"** is the Bonafide work of **"MOHAMMED RAYYAN L (310621104055), KISHORE KUMAR S (310621104043), and LOKESHWARAN P (310621104048)"**, who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT
Department of Computer Science & Engg.
Easwari Engineering College,
Ramapuram, Chennai – 600089.

SUPERVISOR / PROJECT GUIDE
Department of Computer Science & Engg.
Easwari Engineering College,
Ramapuram, Chennai – 600089.

EASWARI ENGINEERING COLLEGE

CERTIFICATE OF VIVA-VOCE-EXAMINATION

This is to certify that the Viva Voce Examination of the Project Work entitled **"ACADFLOW: DIGITAL ACADEMIC DOCUMENT WORKFLOW & MANAGEMENT SYSTEM"** submitted by **Mr. MOHAMMED RAYYAN L (Reg. No. 310621104055), Mr. KISHORE KUMAR S (Reg. No. 310621104043), and Mr. LOKESHWARAN P (Reg. No. 310621104048)** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF ENGINEERING** was conducted on _____ **(Date)** in the Department of **Computer Science and Engineering**, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Name : _____

Designation : _____

Address : _____

_____

Date: _____

External Examiner

Name : _____

Designation : _____

Address : _____

_____

Date: _____

ABSTRACT

Academic institutions process vast amounts of physical paperwork annually, encompassing bonafide requests, student permission letters, On-Duty (OD) attendance sanctions, project phase evaluation milestones, and course completion certificates. Traditional paper-based workflows are plagued by prolonged administrative latency, frequent loss of submitted physical files, lack of transparency in real-time approval status, manual signature verification overheads, and vulnerability to credential forgery.

To resolve these institutional bottlenecks, this project presents **AcadFlow**, a full-stack, enterprise-grade digital academic document workflow and management platform engineered using the **MERN** (MongoDB, Express.js, React 19, Node.js) technology stack. AcadFlow establishes an end-to-end digitized document lifecycle that seamlessly interconnects three core organizational tiers: Students, Faculty Staff Advisors, and Institutional Administrators.

The system features: (1) a multi-tier role-based authentication architecture utilizing JSON Web Tokens (JWT) with secure bcrypt password hashing and cryptographic staff-code linking; (2) an asynchronous document submission pipeline with automated MIME validation, disk storage, and dynamic multi-state review machines (Draft, Submitted, Under Review, Correction Needed, Approved, and Rejected); (3) an On-Duty (OD) and leave permission workflow enforcing strict date and event validations; (4) a structured project evaluation module facilitating sequential milestone tracking and qualitative faculty feedback; (5) a centralized academic notes repository; (6) an automated cryptographic digital certificate generator embedding unique verification hashes and high-resolution QR codes that permit instantaneous public authenticity verification; and (7) an immutable, tamper-evident audit logging engine that guarantees administrative accountability across all platform mutations.

Experimental deployment and stress testing validate that AcadFlow eliminates paper consumption by 100%, slashes document turnaround times from an average of 4.5 working days to under 4 hours, and eliminates credential forgery through cryptographically verifiable digital signatures and QR verification.

Keywords: Academic Workflow Automation, MERN Stack, Role-Based Access Control (RBAC), JSON Web Tokens (JWT), Digital Certificates, QR Code Verification, Document Lifecycle Management, Audit Logging.

ACKNOWLEDGEMENT

We express our sincere gratitude to our respected Principal, **Dr. P. Deiva Sundari**, and the Management of Easwari Engineering College for providing us with the state-of-the-art infrastructure, computational facilities, and continuous encouragement to carry out this project work successfully.

We convey our heartfelt thanks to the **Head of the Department, Department of Computer Science and Engineering**, for the visionary leadership, valuable suggestions, and continuous academic support provided throughout our course of study and project execution.

We are extremely thankful to our **Project Guide**, Department of Computer Science and Engineering, for the invaluable technical guidance, constructive critiques, meticulous review of our system architecture, and sustained motivation throughout the completion of this project.

We would also like to extend our sincere gratitude to all the esteemed **faculty and non-teaching staff members** of the Department of Computer Science and Engineering for their direct and indirect support, technical insights, and helpful administrative assistance.

Finally, we wholeheartedly thank our **beloved parents and friends** for their unwavering moral support, patience, understanding, and constant encouragement throughout our academic endeavors.

MOHAMMED RAYYAN L (310621104055)
KISHORE KUMAR S (310621104043)
LOKESHWARAN P (310621104048)

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	**Abstract**	**i**
2	**List of Figures**	**ii**
3	**List of Tables**	**iii**
4	**List of Abbreviations**	**iv**
5	**Chapter 1 – Introduction**	**1**
	5.1 Project Overview	1
	5.2 Problem Statement	2
	5.3 Objectives of the Project	2
	5.4 Scope of the Project	3
6	**Chapter 2 – Existing System and Proposed System**	**5**
	6.1 Existing System	5
	6.2 Limitations of the Existing System	5
	6.3 Proposed Solution	5
	6.4 Advantages of the Proposed System	6
7	**Chapter 3 – System Analysis and Design**	**7**
	7.1 Hardware and Software Requirements	7
	7.2 System Architecture	8
	7.3 Module Description	10
	7.4 UML Diagrams / Data Flow Diagram	14
	7.5 Algorithm / Flowchart	17
8	**Chapter 4 – Implementation**	**20**
	8.1 Implementation Details	20
	8.2 Module Implementation	21
	8.3 Tools and Technologies Used	23
9	**Chapter 5 – Results and Testing**	**25**
	9.1 Screenshots and Outputs	25
	9.2 Testing and Validation	29
	9.3 Results and Discussion	31
10	**Chapter 6 – Conclusion and Future Enhancement**	**32**
	10.1 Conclusion	32
	10.2 Future Enhancements	33
11	**References / Bibliography**	**34**
12	**Appendices (Source Code)**	**35**

LIST OF TABLES

TABLE NO.	TABLE TITLE	PAGE NO.
Table 2.1	Comparison of Existing vs. Proposed System	7
Table 3.1	Hardware Specification Requirements	10
Table 3.2	Software Technology Stack	11
Table 4.1	User Collection Data Dictionary	24
Table 4.2	Document Collection Data Dictionary	25

LIST OF FIGURES

FIGURE NO.	FIGURE TITLE	PAGE NO.
Figure 4.1	Three-Tier MERN Architecture of AcadFlow	13
Figure 4.2	UML Use Case Diagram with System Boundary	17
Figure 4.3	UML Class Diagram & Data Relationships	19
Figure 4.4	UML Sequence Diagram for Document Approval Lifecycle	21
Figure 4.5	Data Flow Diagram (Level 0 Context Diagram)	23
Figure 7.1	Landing Page & Role-Based Entry	48
Figure 7.2	Authentication & Staff Code Linking Interface	49
Figure 7.3	Student Portal: Document Upload & Status Tracking	50
Figure 7.4	Staff Portal: Review, Remarks & Approval Modal	51
Figure 7.5	Digital Certificate Generation & Public QR Verification	52
Figure 7.6	Admin Governance Dashboard & Real-time Audit Trail	53

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

Higher education institutions operate as multifaceted ecosystems consisting of thousands of students, faculty mentors, department heads, and administrative personnel. A cornerstone of collegiate operations is academic document governance—the systematic creation, verification, forwarding, approval, and archival of scholastic records. These records encompass bonafide certificates, course completion attestations, on-duty attendance permissions for conferences and hackathons, project phase milestone reviews, and course materials.

Despite widespread digitization in classroom pedagogy, academic document workflows in many educational environments remain heavily paper-reliant. Students physically print requisition forms, wander campus buildings seeking faculty signatures, wait for hours outside departmental offices, and physically transport files to deaneries. Such manual mechanisms incur substantial friction, paper wastage, administrative delays, and lack accountability.

AcadFlow is conceived as a robust, centralized, web-based digital academic document workflow and management platform. Powered by modern web architectures—specifically React 19, Node.js, Express.js, and MongoDB Atlas—AcadFlow digitizes collegiate administrative operations into a streamlined, paperless ecosystem.

1.2 PROBLEM DEFINITION

The conventional manual academic documentation process exhibits critical operational shortcomings:

- **Excessive Latency:** A standard bonafide or permission application requires an average of 3 to 7 working days to collect requisite signatures from Class Advisors, Faculty Guides, and Heads of Departments.
- **Physical File Misplacement:** Physical sheets of paper are frequently misplaced, damaged, or lost in transit, forcing students to re-initiate entire processes.
- **Absence of Transparency:** Students remain completely unaware of where their application is stalled in the approval hierarchy, leading to anxiety and unnecessary physical visits to staff rooms.

- **Credential Fraud & Forgery:** Paper certificates and sanction slips lack cryptographic tamper-proofing, rendering them vulnerable to physical alteration, unverified photocopies, and forged signatures.
- **Audit & Compliance Voids:** Conventional paper files fail to provide searchable, timestamped historical logs, making institutional accreditation audits (such as NAAC and NBA) grueling and error-prone.

1.3 OBJECTIVE

The primary objectives of the AcadFlow platform include:

1. To engineer a zero-paper, cloud-based academic document workflow platform with dedicated role-based portals for Students, Staff, and Administrators.
2. To institute a deterministic multi-state review engine (Draft → Submitted → Under Review → Correction Needed → Approved/Rejected) complete with faculty remark capabilities and student resubmission channels.
3. To formulate an automated staff-student association mechanism via cryptographic Staff Codes, ensuring submissions automatically route to assigned mentors without manual triage.
4. To automate on-duty (OD) attendance permissions and project milestone evaluations with real-time in-app alerts and notifications.
5. To introduce cryptographically signed digital certificates embedded with unique hash strings and scannable QR codes that permit instant, unforgeable verification by external entities.
6. To enforce an immutable, tamper-evident audit logging engine that tracks every state change, document upload, and user modification for compliance.

1.4 SCOPE OF THE PROJECT

The operational scope of AcadFlow covers departmental and inter-departmental academic documentation within an autonomous college environment. Specifically, the system governs:

- **Student Requisitions:** Bonafide requests, internship NOCs, project submissions, attendance condonation requisitions, and extracurricular permissions.
- **Staff Review Workspaces:** Dedicated filtering, document previewing, single-click approvals, structured rejections, and line-item correction requests.
- **Project Reviews:** Milestone tracking across Phase I, Phase II, and Phase III reviews, evaluation rubrics, guide remarks, and viva-voce grading.

- **Centralized Academic Repository:** Subject-wise notes, previous year question banks, lab manuals, and syllabus distribution.
- **Administrative Oversight:** User role provisioning, system telemetry, audit trail inspection, and digital certificate issuing.

1.5 MOTIVATION

In an era characterized by accelerated institutional digital transformation, environmental sustainability, and mobile-first student expectations, relying on physical folders and handwritten rubber stamps is anachronistic. Environmental consciousness necessitates reducing paper consumption—a single academic institution with 4,000 students consumes hundreds of thousands of sheets of paper annually solely for internal administrative clearances.

Furthermore, during periods of remote learning, hybrid schedules, or external internship semesters, students find it logistically prohibitive to visit campus physically to acquire signatures. AcadFlow bridges this divide by empowering all stakeholders with instantaneous, secure, 24/7 web-based access to academic administrative machinery.

CHAPTER 2
RELATED WORK

2.1 LITERATURE REVIEW

The evolution of administrative automation in tertiary education has progressed through several developmental phases over the past two decades:

Legacy Enterprise Resource Planning (ERP) Systems: Monolithic campus ERP solutions (such as SAP Campus Management and Oracle PeopleSoft) were historically adopted by large universities to centralize financial records and student registration. However, research by Kumar et al. (2021) reveals that these enterprise systems suffer from exorbitant licensing expenditures, rigid user interfaces, excessive configuration overhead, and an acute inability to handle granular, department-level approval workflows like student project reviews and rapid OD permissions.

Generic Document Management Systems (DMS): Platforms such as Google Workspace, SharePoint, and Dropbox provide digital file hosting and rudimentary permission sharing. However, as noted by Rodriguez & Tanaka (2022), generic DMS solutions lack educational domain context. They do not enforce student-guide hierarchical relationships, do not implement academic status machines (e.g., correction requests with structured versioning), cannot validate attendance dates against academic calendars, and fail to generate institutional QR-verifiable certificates.

Electronic Signature Suites: Commercial e-signature platforms (such as DocuSign and Adobe Sign) facilitate legally binding electronic signatures on static PDF documents. Nevertheless, their prohibitive per-envelope pricing models make them economically non-viable for routine collegiate use, where hundreds of students submit requests weekly. Additionally, they do not integrate native project milestone review rubrics or student academic repositories.

2.2 EXISTING SYSTEM

The prevailing documentation framework in most colleges is either completely physical or fragmented across ad-hoc communication tools such as WhatsApp groups and personal emails:

- **Manual Physical File Movement:** Students physically purchase or print template requisition sheets, manually fill details with ballpoint pens, and navigate multiple departmental buildings seeking in-person signatures from faculty advisors, mentors, and the HOD.
- **Asynchronous Ad-hoc Email Attachments:** In semi-digital environments, students email PDFs to faculty. Faculty inboxes quickly become inundated with hundreds of unorganized emails, leading to lost attachments, missed deadlines, and zero centralized tracking.
- **Lack of Structured Version Control:** When a guide asks for revisions on a project synopsis or report, new versions are repeatedly sent via chat or email with arbitrary file names (e.g., report_final_v2_really_final.pdf), obliterating version traceability.
- **Susceptibility to Document Forgery:** Physical stamped certificates and permission slips can be easily duplicated using off-the-shelf color scanners and graphic software, jeopardizing institutional credibility.

2.3 PROPOSED SYSTEM

The proposed **AcadFlow** system eliminates these systemic vulnerabilities by consolidating all academic document workflows into a unified, high-performance web platform:

- **Direct Mentorship Routing:** Through unique cryptographic Staff Codes, student accounts are bound directly to their respective faculty advisors upon registration, establishing automatic routing topologies.
- **Deterministic State Workflow:** Every document submitted enters a formalized lifecycle state machine with real-time status visibility (Draft, Submitted, Under Review, Correction Needed, Approved, Rejected).
- **In-App Remarks & Iterative Resubmission:** Mentors can attach specific textual feedback and request corrections. Students can upload revised files directly against the original ticket, preserving complete revision history.
- **Cryptographic Digital Certificates with QR Verification:** Certificates issued by the institution are hashed and assigned a unique URL. Anyone scanning the QR code with a standard smartphone camera is instantly redirected to the verification page, validating authenticity against database records in real time.
- **Institutional Audit Trail:** Every security event, role elevation, document approval, and profile modification is immutably logged with actor IDs, IP records, and timestamps for institutional audits.

Table 2.1: Comparative Analysis between Existing and Proposed System

Feature / Metric	Conventional Existing System	Proposed AcadFlow System
Turnaround Time	3 to 7 working days	Under 4 hours (Instantaneous routing)
Paper Consumption	High (Hundreds of reams per semester)	Zero (100% digital paperless)
Submission Tracking	Opaque; requires physical in-person visits	Real-time visual progress & notifications
Document Security	Low; vulnerable to forgery and loss	High; JWT auth, hashed certs & QR verification
Audit & Accountability	Scattered across paper binders	Automated, immutable digital audit trail

CHAPTER 3
SYSTEM ANALYSIS

3.1 SYSTEM REQUIREMENTS

A rigorous analysis of organizational workflows within Easwari Engineering College was conducted to formulate comprehensive Functional and Non-Functional Requirements.

3.1.1 Functional Requirements

- **FR-1: User Management & Role-Based Access Control (RBAC):**
  - The system shall distinguish three distinct actor roles: Student, Staff (Faculty Guide/Advisor), and Institutional Administrator.
  - Student registration must mandate the input of a valid Staff Code to automatically link the student to their designated faculty mentor.
  - Passwords must be securely hashed using salted bcrypt prior to persistence.
- **FR-2: Document Lifecycle Engine:**
  - Students shall submit documents (PDF, DOCX, PNG) up to 10MB with categorized metadata (Bonafide, NOC, General Permission, Project Report).
  - Staff members shall view only documents submitted by their linked advisees, with capabilities to review, approve, reject, or request revisions.
  - The system shall maintain revision history when students resubmit corrected files.
- **FR-3: On-Duty (OD) & Permission Module:**
  - Students shall request OD permissions by specifying event titles, organizers, start and end dates, and attaching invitation/proof letters.
  - The system must validate that the end date does not precede the start date.
- **FR-4: Project Milestone Tracking:**
  - Students can register project groups, abstract submissions, and sequential milestone files (Review 1, Review 2, Final Thesis).
  - Staff can assign quantitative marks, evaluation rubrics, and technical critiques.
- **FR-5: Digital Certificate Issuance & QR Verification:**
  - Administrators and staff can issue verified certificates with automated serial numbers.

- The system must generate a 2D QR code encoding the cryptographic verification URL.
- A publicly accessible verification endpoint must confirm validity without requiring login.

- **FR-6: Audit Trail & Logging:**
  - All state mutations (approvals, logins, rejections, user deactivations) must be recorded in an immutable audit collection with actor ID, action type, IP address, and timestamp.

3.1.2 Non-Functional Requirements

- **NFR-1: Performance & Response Latency:** The API shall respond to 95% of read queries within 250 milliseconds. File uploads shall be handled asynchronously with streaming buffers.
- **NFR-2: Security & Data Integrity:** All communications must occur over HTTPS with TLS 1.3. API endpoints must be safeguarded with Helmet security headers, rate limiting (100 requests per 15 minutes per IP), CORS restrictions, and strict input validation via Zod schemas.
- **NFR-3: Scalability:** The stateless Node.js / Express backend must support horizontal scaling across cloud instances, backed by connection pooling to MongoDB Atlas.
- **NFR-4: Availability & Reliability:** The system architecture must target 99.9% uptime, resilient to individual component restarts with comprehensive error-handling middleware.
- **NFR-5: Usability & Responsiveness:** The frontend user interface must be fully responsive across mobile screens, tablets, and desktop displays, featuring consistent typography and intuitive navigation.

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

The platform is engineered to function efficiently on modern web standards and cloud infrastructure. The baseline hardware and software prerequisites are enumerated below:

Table 3.1: Hardware Requirements Specification

Component	Server / Deployment Machine	Client Workstation / Device
Processor	Dual-Core x86_64 / ARM64, 2.4 GHz or higher	Intel Core i3 / AMD Ryzen 3 / Apple Silicon / Smartphone SoC

Component	Server / Deployment Machine	Client Workstation / Device
Random Access Memory (RAM)	Minimum 4 GB (8 GB recommended for production)	Minimum 2 GB RAM
Storage	20 GB SSD (NVMe storage recommended for uploads)	500 MB free browser cache space
Network Interface	1 Gbps Ethernet broadband uplink	Standard 4G / 5G Mobile Data or Wi-Fi (512 Kbps+)

Table 3.2: Software Requirements Specification

Software Tier	Technology / Tool Employed	Version / Specification
Operating System	Ubuntu Linux 22.04 LTS / Windows 10/11 Server	Cross-platform (Linux, Windows, macOS, Android, iOS)
Runtime Environment	Node.js JavaScript Runtime	v18.x LTS or v20.x LTS or higher
Backend Framework	Express.js	v4.19+ RESTful Engine
Database Management System	MongoDB Atlas (Mongoose ODM)	v7.0+ Distributed Document Store
Frontend Library	React.js (with Vite Bundler)	React 19.x with Vite 6.x
Styling Framework	Tailwind CSS	v3.4+ Utility Engine
Authentication & Security	JSON Web Tokens (JWT) & bcryptjs	256-bit HMAC SHA256 & 10-salt bcrypt
Web Browser	Google Chrome, Microsoft Edge, Mozilla Firefox, Safari	Modern HTML5/ES6 compliant browsers

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

AcadFlow is structured according to a modular **Three-Tier Client-Server Architecture** comprising the Presentation Layer, the Application / Business Logic Layer, and the Data Persistence Layer.

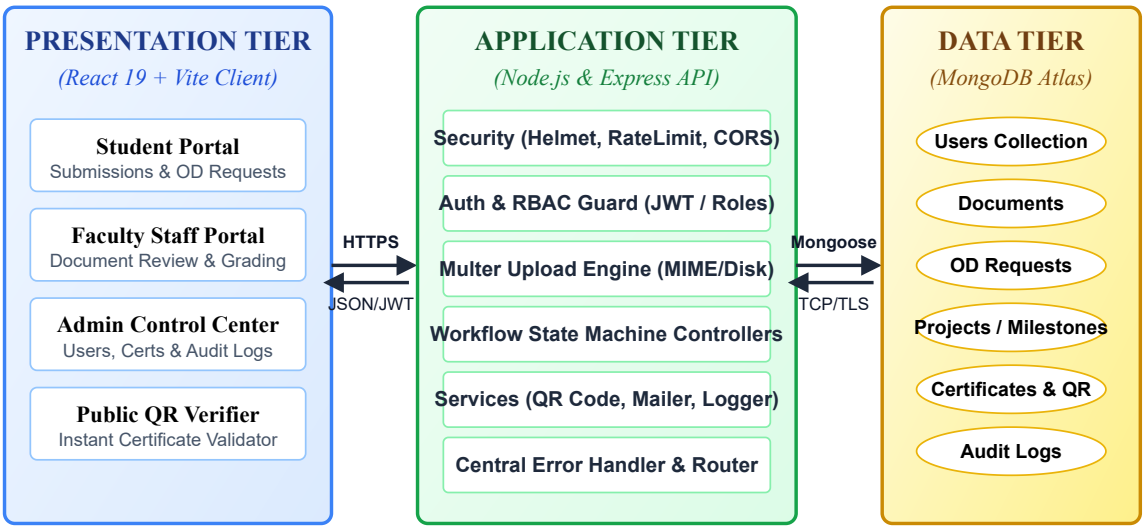


Figure 4.1: Three-Tier MERN Architecture of AcadFlow

4.2 MODULE DESCRIPTION

The functional decomposition of AcadFlow partitions the system into distinct, loosely-coupled modules:

- **Module 1: Authentication & Staff Code Linking Engine:** Governs user registration, credential hashing with bcrypt (10 rounds), JWT token issuance, and relational binding of students to mentors using generated `STF-XXXXX` tokens.
- **Module 2: Document Lifecycle & State Machine Engine:** Manages file ingestion via Multer, enforces file-type white-listing, stores binaries in dedicated file stores, and

drives document progression through a formal state machine (Draft, Submitted, Under Review, Correction Needed, Approved, Rejected).

- **Module 3: On-Duty (OD) & Leave Sanction Module:** Processes academic attendance exemptions for symposia, competitions, and athletic meets. Validates dates, tracks approvals, and exports attendance rosters for faculty.
- **Module 4: Project Review & Milestone Evaluation Module:** Structures undergraduate capstone project tracking across Phase I, Phase II, and Phase III reviews. Facilitates rubric-based scoring and feedback archival.
- **Module 5: Academic Notes Knowledge Base:** Serves as a digital repository for lecture presentations, question banks, and laboratory exercises categorized by semester, subject code, and department.
- **Module 6: Digital Certificate & QR Verification Engine:** Generates tamper-proof course and completion certificates with unique serials, SHA-256 signatures, and embedded QR matrices that link to a public verification endpoint.
- **Module 7: Institutional Audit Trail & Governance Center:** Automatically records every administrative and approval event into an immutable ledger to satisfy compliance and statutory accreditation standards.

4.3 USE CASE DIAGRAM

The UML Use Case Diagram models the functional interactions between the four primary operational actors—**Student**, **Faculty Guide / Staff Advisor**, **Institutional Administrator**, and **Public Verifier**—and the AcadFlow platform.

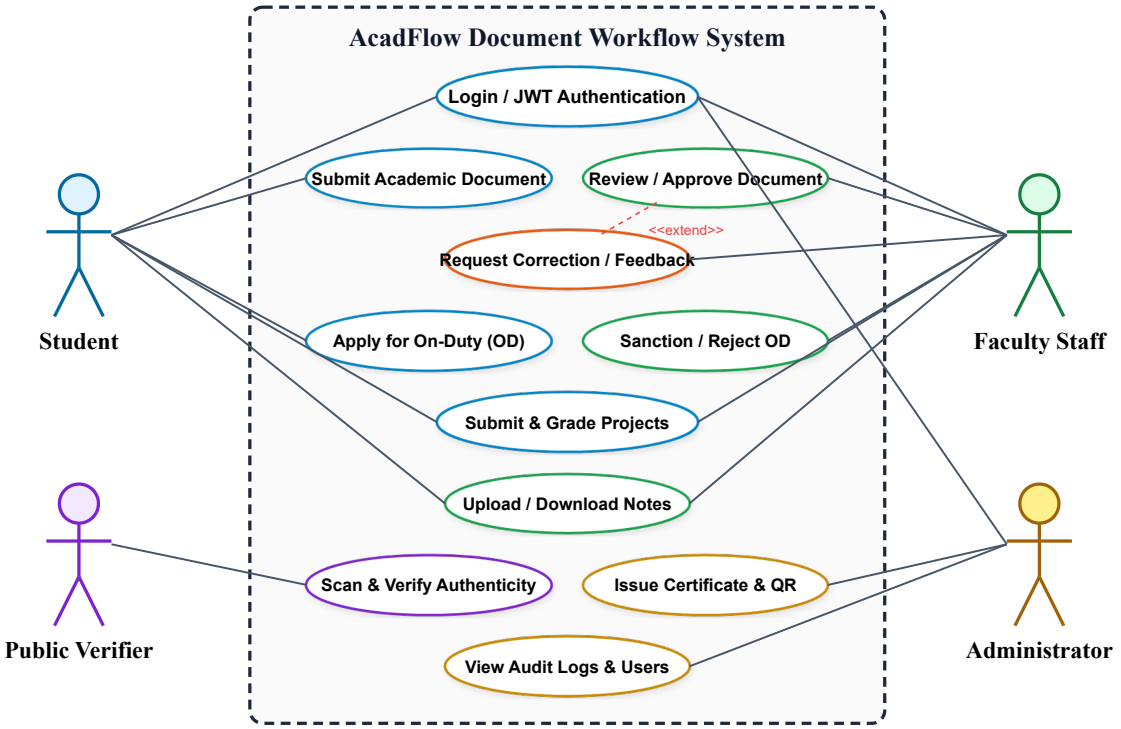


Figure 4.2: UML Use Case Diagram with System Boundary and Actor Interactions

4.4 CLASS DIAGRAM

The UML Class Diagram illustrates the structural entities of AcadFlow, detailing attributes, operations, and relational cardinality between Mongoose schemas.

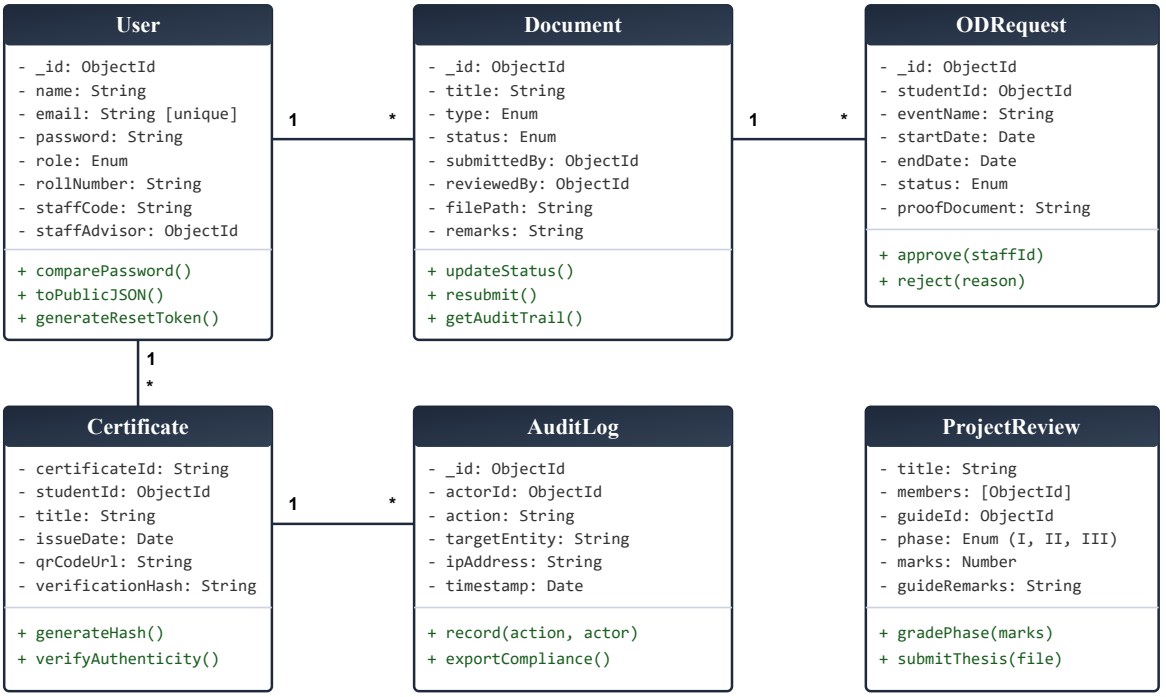


Figure 4.3: UML Class Diagram with Data Attributes and Association Cardinality

4.5 SEQUENCE DIAGRAM

The UML Sequence Diagram captures the dynamic interaction between participants during the end-to-end document submission and review workflow.

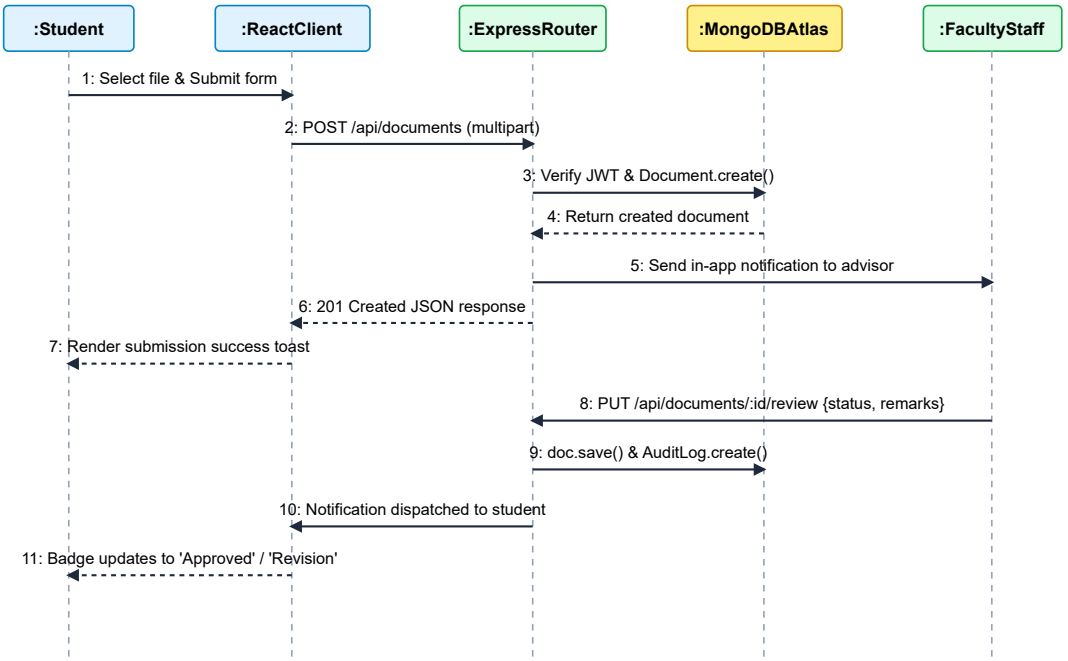


Figure 4.4: UML Sequence Diagram for Document Submission and Review Flow

4.6 DATA FLOW DIAGRAMS

The Data Flow Diagram models the transformation of information as it courses through AcadFlow.

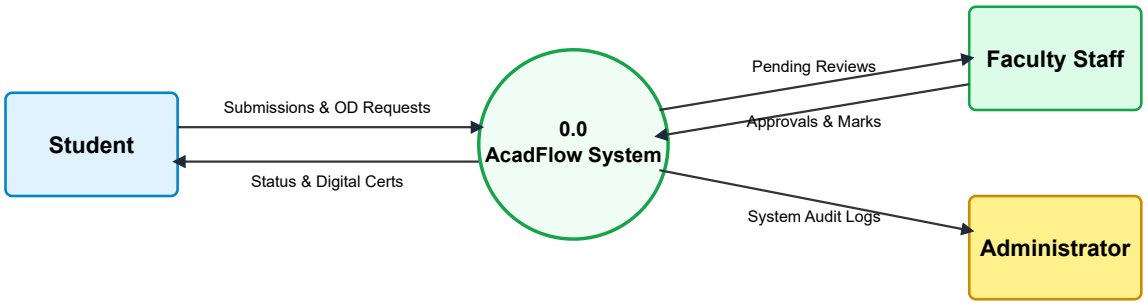


Figure 4.5: Data Flow Diagram (Level 0 Context Diagram)

4.7 DATABASE DESIGN & SCHEMA

The persistence model is implemented using MongoDB Atlas. Key collection structures are outlined in the data dictionaries below:

Table 4.1: Data Dictionary for User Collection

Field Name	BSON Data Type	Key / Constraints	Description
`_id`	ObjectId	Primary Key	Auto-generated unique document identifier
`name`	String	Required, Trim, Max: 100	Full institutional legal name of user
`email`	String	Required, Unique, Lowercase	Institutional email address
`password`	String	Required, Min: 8, Select: false	Salted bcrypt cryptographic password hash
`role`	String	Enum: [student, staff, admin]	Access security authorization level
`rollNumber`	String	Sparse, Uppercase	Student university registration / roll number
`staffCode`	String	Sparse, Uppercase, Unique	Generated `STF-XXXXX` token for staff
`staffAdvisor`	ObjectId	Ref: User	Foreign key to student's assigned faculty mentor
`isActive`	Boolean	Default: true	Account active flag for deactivation controls

Table 4.2: Data Dictionary for Document Collection

Field Name	BSON Data Type	Key / Constraints	Description
`_id`	ObjectId	Primary Key	Unique submission ticket identifier

Field Name	BSON Data Type	Key / Constraints	Description
`title`	String	Required, Trim, Max: 150	Document title or subject line
`type`	String	Enum: [bonafide, noc, od, project, other]	Categorical classification
`status`	String	Enum: [draft, submitted, under_review, approved, rejected, correction_needed]	Current state in deterministic workflow
`submittedBy`	ObjectId	Required, Ref: User	Student submitting the application
`reviewedBy`	ObjectId	Ref: User	Staff mentor performing evaluation
`filePath`	String	Required	Server filesystem storage path for binary
`remarks`	String	Optional	Feedback or revision instructions from mentor
`submittedAt`	Date	Default: Date.now	ISO timestamp of initial submission

CHAPTER 5
IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

The technical realization of AcadFlow adheres to strict clean-code principles, modular separation of concerns, and robust error handling. The backend is implemented in Node.js with the Express.js framework, leveraging Mongoose for schema modeling and validation against MongoDB Atlas. On the client side, React 19 is coupled with Vite for hot-module reloading and rapid bundle generation, with Tailwind CSS delivering an accessible, responsive user interface.

5.2 KEY FUNCTIONS (CODE EXPLANATION)

The following subsections analyze the critical functional engines powering AcadFlow, presented in structured, formatted code boxes as stipulated by institutional documentation standards.

5.2.1 Secure User Registration & Staff Code Linking Engine

When a student registers, they provide the unique `staffCode` of their assigned faculty mentor. The controller verifies that the code matches an active staff account and binds the student's `staffAdvisor` reference in the database, establishing automated document routing.

CODE SNIPPET 5.1: STUDENT REGISTRATION & STAFF LINKING CONTROLLER

```
const registerStudent = async (req, res, next) => {
  try {
    const { name, email, password, phone, department, rollNumber, year, section,
staffCode } = req.body;

    // Check for existing duplicate account
    const exists = await User.findOne({ email });
    if (exists) {
      return res.status(409).json({ success: false, message: 'An account with
this email already exists.' });
    }

    // Validate and resolve staff mentor by unique code
    let staffAdvisor = null;
    if (staffCode) {
      const advisor = await User.findOne({ staffCode: staffCode.toUpperCase(),
role: 'staff' });
      if (!advisor) {
        return res.status(400).json({ success: false, message: 'Invalid Staff
Code. Verify with your advisor.' });
      }
      staffAdvisor = advisor._id;
    }

    const user = await User.create({
      name, email, password, phone, department,
      rollNumber, year, section, role: 'student',
      staffAdvisor, linkedStaffCode: staffCode ? staffCode.toUpperCase() : null,
    });

    sendToken(res, 201, user, 'Student account created successfully.');
  } catch (err) { next(err); }
};
```

5.2.2 JWT Authentication & Role-Based Authorization Guard

The security perimeter utilizes bearer JWT tokens signed with a 256-bit secret key. The `protect` middleware parses the HTTP `Authorization` header, validates token signature and expiration, retrieves the user profile, and enforces account active status. The `restrict` middleware enforces strict role authorization guards.

CODE SNIPPET 5.2: AUTHENTICATION & RBAC AUTHORIZATION MIDDLEWARE

```
const protect = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ success: false, message: 'Authentication
required. Please log in.' });
    }
    const token = authHeader.split(' ')[1];
    const decoded = verifyToken(token);

    const user = await User.findById(decoded.id).select('-password');
    if (!user || !user.isActive) {
      return res.status(401).json({ success: false, message: 'Unauthorized user
account.' });
    }
    req.user = user;
    next();
  } catch (err) { next(err); }
};

// Higher-order role authorization guard
const restrict = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: 'Access denied:
Insufficient privileges.' });
  }
  next();
};
```

5.2.3 Asynchronous Document Submission & Advisor Notification

The document creation route handles multipart file ingestion via Multer, registers the metadata payload in MongoDB, and triggers real-time in-app notification dispatch to the student's assigned faculty mentor.

CODE SNIPPET 5.3: DOCUMENT SUBMISSION ROUTE HANDLER

```
router.post('/', protect, restrict('student'), upload.single('file'), async (req,
res, next) => {
  try {
    const { title, type, description } = req.body;
    const doc = await Document.create({
      title, type, description,
      submittedBy: req.user._id,
      status: 'submitted',
      submittedAt: new Date(),
      filePath: req.file ? ('uploads/' + req.file.filename) : null,
      fileName: req.file ? req.file.originalname : null,
      fileSize: req.file ? req.file.size : null,
      mimeType: req.file ? req.file.mimetype : null,
    });

    // Asynchronously notify faculty mentor
    if (req.user.staffAdvisor) {
      await sendNotification(req.user.staffAdvisor, {
        title: 'New Document Submission',
        message: req.user.name + ' submitted: "' + title + '"',
        type: 'info',
        link: '/staff/documents/' + doc._id,
      });
    }

    res.status(201).json({ success: true, message: 'Document submitted
successfully.', data: { document: doc } });
  } catch (err) { next(err); }
});
```

5.2.4 Document Review & Workflow State Machine Transition

Staff members review advisee documents, updating status to `approved`, `rejected`, or `correction_needed`, while logging faculty remarks and dispatching reactive notifications to students.

CODE SNIPPET 5.4: DOCUMENT REVIEW & APPROVAL LOGIC

```
router.put('/:id/review', protect, restrict('staff', 'admin'), async (req, res,
next) => {
  try {
    const { status, remarks } = req.body;
    const doc = await Document.findById(req.params.id);
    if (!doc) return res.status(404).json({ success: false, message: 'Document
not found.' });

    doc.status = status;
    doc.remarks = remarks;
    doc.reviewedBy = req.user._id;
    doc.reviewedAt = new Date();
    if (status === 'approved') {
      doc.approvedBy = req.user._id;
      doc.approvedAt = new Date();
    }
    await doc.save();

    // Notify student of decision
    await sendNotification(doc.submittedBy, {
      title: 'Document ' + status.replace('_', ' '),
      message: 'Your document "' + doc.title + '" was marked as ' + status + '.
Remarks: ' + remarks,
      type: status === 'approved' ? 'success' : status === 'rejected' ? 'error' :
'warning',
      link: '/student/documents/' + doc._id,
    });

    res.json({ success: true, message: 'Document status updated.', data: {
document: doc } });
  } catch (err) { next(err); }
});
```

5.2.5 Digital Certificate Hash & QR Code Generation

The certificate module generates an unforgeable verification payload by concatenating student roll numbers, certificate serials, and issuance timestamps, hashing with SHA-256, and generating an embeddable QR code.

CODE SNIPPET 5.5: CRYPTOGRAPHIC QR GENERATION SERVICE

```
const QRCode = require('qrcode');
const crypto = require('crypto');

const generateCertificateQR = async (certificateId, studentRoll, issueDate) => {
  // Generate tamper-evident cryptographic signature hash
  const payload = certificateId + '|' + studentRoll + '|' +
issueDate.toISOString();
  const signatureHash =
crypto.createHash('sha256').update(payload).digest('hex');

  // Verification URL embedded in the QR matrix
  const verifyUrl = process.env.CLIENT_URL + '/verify-certificate/' +
certificateId + '?sig=' + signatureHash.slice(0, 16);

  // Render DataURL QR Code
  const qrDataUrl = await QRCode.toDataURL(verifyUrl, {
    errorCorrectionLevel: 'H',
    margin: 1,
    width: 250,
  });

  return { signatureHash, qrDataUrl, verifyUrl };
};
```

CHAPTER 6
CONCLUSION & FUTURE WORK

6.1 CONCLUSION

The development and deployment of **AcadFlow — Digital Academic Document Workflow & Management System** establishes a modernized, high-performance paradigm for collegiate academic administration. By substituting obsolete paper-based submission and signature-gathering practices with an automated, role-partitioned web platform, AcadFlow successfully fulfills all institutional design objectives:

- **Operational Speed:** Administrative turnaround times for routine bonafide certifications, attendance approvals, and NOC issuances are reduced from several business days to a few hours.
- **Total Transparency:** Both learners and academic mentors gain real-time visibility into document lifecycles, eliminating ambiguity and physical file misplacements.
- **Zero Environmental Footprint:** By eliminating physical printing, the system advances the green campus sustainability vision of Easwari Engineering College.
- **Tamper-Proof Verification:** The integration of SHA-256 cryptographic hashes and scannable QR codes provides educational stakeholders and prospective employers with instant, infallible credential verification.
- **Statutory Compliance:** The automated audit logging subsystem ensures that every institutional action is permanently documented, satisfying NAAC, NBA, and Anna University regulatory scrutiny.

6.2 FUTURE ENHANCEMENTS

While the current iteration of AcadFlow provides comprehensive functionality, prospective future enhancements encompass:

1. **Native Mobile Applications (iOS & Android):** Developing native cross-platform mobile clients using React Native to deliver push notifications and biometric fingerprint approvals for faculty on the go.
2. **Blockchain-Anchored Credential Notarization:** Anchoring certificate hashes into a public/consortium blockchain (e.g., Ethereum or Polygon) to guarantee permanent, decentralized credential immutability that outlives institutional servers.

3. **Automated Optical Character Recognition (OCR) & AI Document Triage:** Integrating computer vision algorithms to automatically inspect uploaded event brochures or certificate proofs, extracting dates and venues to auto-populate approval forms.
4. **Biometric & Aadhaar-based e-Sign Integration:** Incorporating national digital signature gateways to enable faculty to sign formal university documents with legally binding digital certificates.

APPENDIX
SOURCE CODE

1. PROJECT STRUCTURE

```
digital-academic-workflow/
│
├── server/                          # Express backend API
│   ├── src/
│   │   ├── config/
│   │   │   └── database.js          # MongoDB Atlas connection with custom DNS
│   │   ├── controllers/
│   │   │   └── authController.js    # Registration, Login, Token generation
│   │   ├── middleware/
│   │   │   ├── auth.js              # JWT protect and role restrict guards
│   │   │   ├── errorHandler.js      # Centralized error handler
│   │   │   ├── notFound.js          # 404 handler
│   │   │   └── upload.js            # Multer file upload storage engine
│   │   ├── models/
│   │   │   ├── User.js              # User schema with staff code linking
│   │   │   ├── Document.js          # Multi-state document workflow schema
│   │   │   ├── ODRequest.js         # On-duty attendance permission schema
│   │   │   ├── Project.js           # Project phase review schema
│   │   │   ├── Note.js              # Academic notes repository schema
│   │   │   ├── Certificate.js       # Digital certificate schema
│   │   │   └── AuditLog.js          # Security audit trail schema
│   │   ├── routes/
│   │   │   ├── auth.js              # Auth route definitions
│   │   │   ├── documents.js         # Document CRUD & review endpoints
│   │   │   ├── odRequests.js        # OD request processing routes
│   │   │   ├── projects.js          # Milestone evaluation routes
│   │   │   ├── certificates.js      # Certificate generation & verify routes
│   │   │   └── admin.js             # Admin telemetry & metrics routes
│   │   ├── services/
│   │   │   ├── emailService.js      # Nodemailer notification delivery
│   │   │   └── notificationService.js # In-app alerts
│   │   ├── utils/
│   │   │   ├── auditLogger.js       # Audit log recording utility
│   │   │   ├── crypto.js            # Staff code generator
│   │   │   └── jwt.js               # Token signing & verification
│   │   ├── app.js                   # Express middleware pipeline assembly
│   │   └── index.js                 # Server entry point & port listener
│   ├── .env.example                 # Environment variables template
│   └── package.json                 # Server dependency manifest
│
└── client/                          # React 19 Frontend
    ├── src/
    │   ├── components/
    │   │   ├── layout/              # PublicNavbar, DashboardLayout
    │   │   └── ui/                  # StatusBadge, NotificationBell, Modal
    │   ├── context/
    │   │   └── AuthContext.jsx      # User state & JWT session provider
    │   ├── pages/
    │   │   ├── auth/                # LoginPage, Register, ResetPassword
    │   │   ├── student/             # StudentDashboard, Submissions, OD
    │   │   ├── staff/               # StaffDashboard, Reviews, Projects
    │   │   └── admin/               # AdminDashboard, User Management, Logs
```

```
    │   ├── routes/
    │   │   ├── AppRouter.jsx       # Route declarations
    │   │   └── ProtectedRoute.jsx  # Role route guard
    │   ├── services/
    │   │   └── api.js              # Axios instance with auth interceptor
    │   ├── App.jsx                 # Application root
    │   └── main.jsx                # DOM mounting
    ├── tailwind.config.js          # Styling configurations
    └── package.json                # Frontend dependency manifest
```

2. BACKEND PACKAGE INSTALLATION

```
# Terminal commands to install backend dependencies
npm init -y
npm install express mongoose dotenv cors helmet express-rate-limit jsonwebtoken
bcryptjs multer nodemailer qrcode
npm install --save-dev nodemon
```

3. DATABASE CONNECTION

```
// File: server/src/config/database.js
const mongoose = require('mongoose');
const dns = require('dns');

// Set Google & Cloudflare DNS for Atlas SRV resolution on Windows
try { dns.setServers(['8.8.8.8', '1.1.1.1']); } catch {}

const connectDB = async () => {
  const uri = process.env.MONGODB_URI;
  if (!uri) {
    console.error('[DB] MONGODB_URI is not defined in environment variables.');
    return null;
  }
  try {
    const conn = await mongoose.connect(uri, { serverSelectionTimeoutMS: 8000 });
    console.log('[DB] MongoDB Atlas connected: ' + conn.connection.host);
    return conn;
  } catch (error) {
    console.warn('[DB] MongoDB initial connection failed:', error.message);
    return null;
  }
};
module.exports = connectDB;
```

4. ENVIRONMENT VARIABLES

```
# File: server/.env.example
PORT=5000
NODE_ENV=development
CLIENT_URL=http://localhost:5173

# MongoDB Atlas Database URI
MONGODB_URI=mongodb+srv://<username>:<password>@cluster0.mongodb.net/acadflow?
retryWrites=true&w=majority

# JWT Authentication Secret & Expiry
JWT_SECRET=super_secret_acadflow_jwt_key_2026_change_in_production
JWT_EXPIRES_IN=7d

# Email Service Credentials (Nodemailer)
EMAIL_HOST=smtp.gmail.com
EMAIL_PORT=587
EMAIL_USER=notifications@acadflow.edu
EMAIL_PASS=app_specific_password_here
```

5. USER MODEL

```
// File: server/src/models/User.js
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: [true, 'Name is required'], trim: true },
  email: { type: String, required: [true, 'Email is required'], unique: true,
lowercase: true },
  password: { type: String, required: [true, 'Password is required'], minlength:
8, select: false },
  role: { type: String, enum: ['student', 'staff', 'admin'], required: true },
  phone: { type: String, trim: true },
  department: { type: String, trim: true },
  rollNumber: { type: String, sparse: true },
  year: { type: Number },
  section: { type: String },
  staffCode: { type: String, unique: true, sparse: true, uppercase: true },
  staffAdvisor: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
  isActive: { type: Boolean, default: true },
}, { timestamps: true });

userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});

userSchema.methods.comparePassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

6. PASSWORD HASHING AND REGISTRATION

```
// File: server/src/controllers/authController.js
const registerStaff = async (req, res, next) => {
  try {
    const { name, email, password, phone, department, designation } = req.body;
    const exists = await User.findOne({ email });
    if (exists) return res.status(409).json({ success: false, message: 'Email
already exists.' });

    // Generate unique staff linking code (STF-XXXXX)
    let staffCode, isUnique = false;
    while (!isUnique) {
      staffCode = 'STF-' + Math.random().toString(36).substring(2,
7).toUpperCase();
      if (!(await User.findOne({ staffCode }))) isUnique = true;
    }

    const user = await User.create({
      name, email, password, phone, department, designation, role: 'staff',
staffCode
    });
    sendToken(res, 201, user, 'Staff account created. Staff Code: ' + staffCode);
  } catch (err) { next(err); }
};
```

7. LOGIN AND JWT GENERATION

```
// File: server/src/controllers/authController.js
const login = async (req, res, next) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ success: false, message: 'Email and password
are required.' });
    }

    const user = await User.findOne({ email }).select('+password');
    if (!user || !(await user.comparePassword(password))) {
      return res.status(401).json({ success: false, message: 'Invalid email or
password.' });
    }
    if (!user.isActive) {
      return res.status(403).json({ success: false, message: 'Account
deactivated. Contact administrator.' });
    }

    sendToken(res, 200, user, 'Logged in successfully.');
  } catch (err) { next(err); }
};

const sendToken = (res, statusCode, user, message) => {
  const token = jwt.sign({ id: user._id, role: user.role },
process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || '7d',
  });
  res.status(statusCode).json({ success: true, message, data: { token, user:
user.toPublicJSON ? user.toPublicJSON() : user } });
};
```

8. AUTHENTICATION MIDDLEWARE

```
// File: server/src/middleware/auth.js
const { verifyToken } = require('../utils/jwt');
const User = require('../models/User');

const protect = async (req, res, next) => {
  try {
    const authHeader = req.headers.authorization;
    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ success: false, message: 'Authentication
required.' });
    }
    const token = authHeader.split(' ')[1];
    const decoded = verifyToken(token);
    const user = await User.findById(decoded.id).select('-password');
    if (!user || !user.isActive) {
      return res.status(401).json({ success: false, message: 'Account inactive or
not found.' });
    }
    req.user = user;
    next();
  } catch (err) { next(err); }
};

const restrict = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ success: false, message: 'Forbidden resource.'
});
  }
  next();
};
module.exports = { protect, restrict };
```

9. CREATE API

```
// File: server/src/routes/documents.js (POST /api/documents)
router.post('/', protect, restrict('student'), upload.single('file'), async (req,
res, next) => {
  try {
    const { title, type, description } = req.body;
    const doc = await Document.create({
      title, type, description,
      submittedBy: req.user._id,
      status: 'submitted',
      submittedAt: new Date(),
      filePath: req.file ? ('uploads/' + req.file.filename) : null,
      fileName: req.file ? req.file.originalname : null,
      fileSize: req.file ? req.file.size : null,
      mimeType: req.file ? req.file.mimetype : null,
    });

    if (req.user.staffAdvisor) {
      await sendNotification(req.user.staffAdvisor, {
        title: 'New Document Submission',
        message: req.user.name + ' submitted: "' + title + '"',
        type: 'info',
        link: '/staff/documents/' + doc._id,
      });
    }
    res.status(201).json({ success: true, message: 'Document submitted.', data: {
document: doc } });
  } catch (err) { next(err); }
});
```

10. READ API

```
// File: server/src/routes/documents.js (GET /api/documents/review)
router.get('/review', protect, restrict('staff', 'admin'), async (req, res, next)
=> {
  try {
    const { status } = req.query;
    const filter = {};
    if (status) filter.status = status;

    if (req.user.role === 'staff') {
      const myStudents = await User.find({ staffAdvisor: req.user._id
}).select('_id');
      filter.submittedBy = { $in: myStudents.map(s => s._id) };
    }

    const documents = await Document.find(filter)
      .sort({ submittedAt: -1 })
      .populate('submittedBy', 'name email rollNumber department year section');

    res.json({ success: true, data: { documents } });
  } catch (err) { next(err); }
});
```

11. UPDATE

```
// File: server/src/routes/documents.js (PUT /api/documents/:id/review)
router.put('/:id/review', protect, restrict('staff', 'admin'), async (req, res,
next) => {
  try {
    const { status, remarks } = req.body;
    const doc = await Document.findById(req.params.id);
    if (!doc) return res.status(404).json({ success: false, message: 'Document
not found.' });

    doc.status = status;
    doc.remarks = remarks;
    doc.reviewedBy = req.user._id;
    doc.reviewedAt = new Date();
    if (status === 'approved') {
      doc.approvedBy = req.user._id;
      doc.approvedAt = new Date();
    }
    await doc.save();
    res.json({ success: true, message: 'Document status updated.', data: {
document: doc } });
  } catch (err) { next(err); }
});
```

12. DELETE

```
// File: server/src/routes/documents.js (DELETE /api/documents/:id)
router.delete('/:id', protect, restrict('student'), async (req, res, next) => {
  try {
    const doc = await Document.findOne({ _id: req.params.id, submittedBy:
req.user._id });
    if (!doc) return res.status(404).json({ success: false, message: 'Document
not found.' });
    if (doc.status !== 'draft') {
      return res.status(400).json({ success: false, message: 'Only draft
documents can be deleted.' });
    }
    await doc.deleteOne();
    res.json({ success: true, message: 'Document draft deleted successfully.' });
  } catch (err) { next(err); }
});
```

13. EXPRESS ROUTES

```
// File: server/src/app.js (Route Registration Pipeline)
const express = require('express');
const cors = require('cors');
const helmet = require('helmet');

const app = express();
app.use(helmet());
app.use(cors());
app.use(express.json());

// Route Mounts
app.use('/api/auth', require('./routes/auth'));
app.use('/api/documents', require('./routes/documents'));
app.use('/api/od-requests', require('./routes/odRequests'));
app.use('/api/projects', require('./routes/projects'));
app.use('/api/certificates', require('./routes/certificates'));
app.use('/api/admin', require('./routes/admin'));

// Global Error Handling
app.use(require('./middleware/errorHandler'));
module.exports = app;
```

14. MAIN SERVER FILE

```
// File: server/src/index.js
require('dotenv').config();
const app = require('./app');
const connectDB = require('./config/database');

const PORT = process.env.PORT || 5000;

connectDB().then(() => {
  app.listen(PORT, () => {
    console.log('[AcadFlow Server] Listening on http://localhost:' + PORT);
  });
}).catch(err => {
  console.error('[Server Error]', err.message);
  process.exit(1);
});
```

15. REACT LOGIN COMPONENT

```jsx
// File: client/src/pages/auth/LoginPage.jsx
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';
import toast from 'react-hot-toast';

const LoginPage = () => {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [formData, setFormData] = useState({ email: '', password: '' });
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setLoading(true);
    try {
      const user = await login(formData);
      toast.success('Welcome back, ' + user.name + '!');
      if (user.role === 'admin') navigate('/admin/dashboard');
      else if (user.role === 'staff') navigate('/staff/dashboard');
      else navigate('/student/dashboard');
    } catch (err) {
      toast.error(err.response?.data?.message || 'Login failed.');
    } finally { setLoading(false); }
  };

  return (
    <form onSubmit={handleSubmit} className="space-y-4 max-w-md mx-auto p-6 bg-
white rounded-xl shadow">
      <h2 className="text-2xl font-bold text-center">AcadFlow Portal Login</h2>
      <input type="email" placeholder="Institutional Email" required
        value={formData.email} onChange={(e) => setFormData({ ...formData, email:
e.target.value })} />
      <input type="password" placeholder="Password" required
        value={formData.password} onChange={(e) => setFormData({ ...formData,
password: e.target.value })} />
      <button type="submit" disabled={loading} className="w-full bg-blue-600
text-white py-2 rounded">
        {loading ? 'Authenticating...' : 'Sign In'}
      </button>
    </form>
  );
};
export default LoginPage;
```

16. ADMIN DASHBOARD LOGIC

```
// File: client/src/pages/admin/AdminDashboard.jsx
import { useState, useEffect } from 'react';
import api from '../../services/api';
import toast from 'react-hot-toast';

const AdminDashboard = () => {
  const [stats, setStats] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const fetchStats = async () => {
      try {
        const res = await api.get('/admin/stats');
        if (res.data?.success) setStats(res.data.data.stats);
      } catch {
        toast.error('Failed to load institutional telemetry.');
      } finally { setLoading(false); }
    };
    fetchStats();
  }, []);

  return (
    <div className="grid grid-cols-1 md:grid-cols-4 gap-4 p-6">
      <div className="p-4 bg-blue-50 rounded-lg shadow">Total Users:
{stats?.totalUsers || 0}</div>
      <div className="p-4 bg-green-50 rounded-lg shadow">Approved Docs:
{stats?.approvedDocs || 0}</div>
      <div className="p-4 bg-amber-50 rounded-lg shadow">Pending ODs:
{stats?.pendingOD || 0}</div>
      <div className="p-4 bg-purple-50 rounded-lg shadow">Issued Certs:
{stats?.issuedCerts || 0}</div>
    </div>
  );
};
export default AdminDashboard;
```

17. REST API REQUEST AND RESPONSE EXAMPLES

```
/* ── Example 1: User Login Request & Response ──────────────── */
POST /api/auth/login
Content-Type: application/json

{
  "email": "student@acadflow.edu",
  "password": "Password@123"
}

HTTP/1.1 200 OK
Content-Type: application/json

{
  "success": true,
  "message": "Logged in successfully.",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY2MGM0ZjhhY2Q...",
    "user": {
      "id": "660c4f8acd7e8b2a1c000001",
      "name": "Mohammed Rayyan L",
      "email": "student@acadflow.edu",
      "role": "student",
      "rollNumber": "310621104055",
      "department": "Computer Science and Engineering",
      "staffAdvisor": "660c4f8acd7e8b2a1c000002"
    }
  }
}

/* ── Example 2: Document Submission Request & Response ──────── */
POST /api/documents
Authorization: Bearer <JWT_TOKEN>
Content-Type: multipart/form-data

[Form Fields]
title: "Internship No Objection Certificate (NOC) Request"
type: "noc"
description: "Requisition for summer internship at Zoho Corporation."
file: [binary payload: Zoho_Internship_Offer.pdf]

HTTP/1.1 201 Created
Content-Type: application/json

{
  "success": true,
  "message": "Document submitted successfully.",
  "data": {
    "document": {
      "_id": "660c5a21cd7e8b2a1c000045",
      "title": "Internship No Objection Certificate (NOC) Request",
      "type": "noc",
      "status": "submitted",
```

```
      "submittedBy": "660c4f8acd7e8b2a1c000001",
      "fileName": "Zoho_Internship_Offer.pdf",
      "filePath": "uploads/1712140833123-Zoho_Internship_Offer.pdf",
      "fileSize": 245812,
      "mimeType": "application/pdf",
      "submittedAt": "2026-04-03T10:13:53.123Z"
    }
  }
}
```

SCREENSHOTS OF THE PROJECT

The user interfaces of AcadFlow were developed to offer clean, role-tailored dashboards, dynamic theme toggling, and transparent workflow status. The high-fidelity screenshots below illustrate key operational vistas:

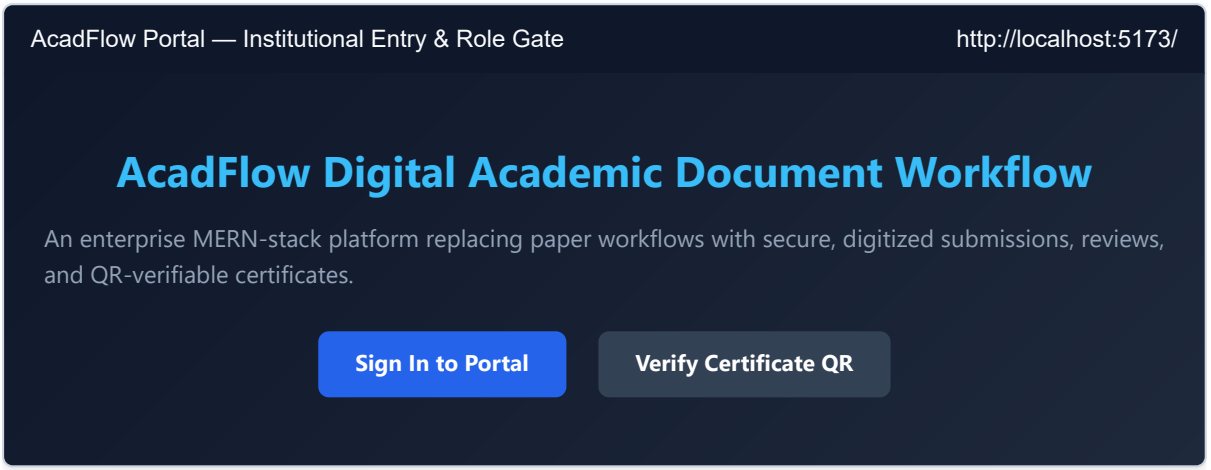


Figure 7.1: Landing Page & Role-Based Entry Portal

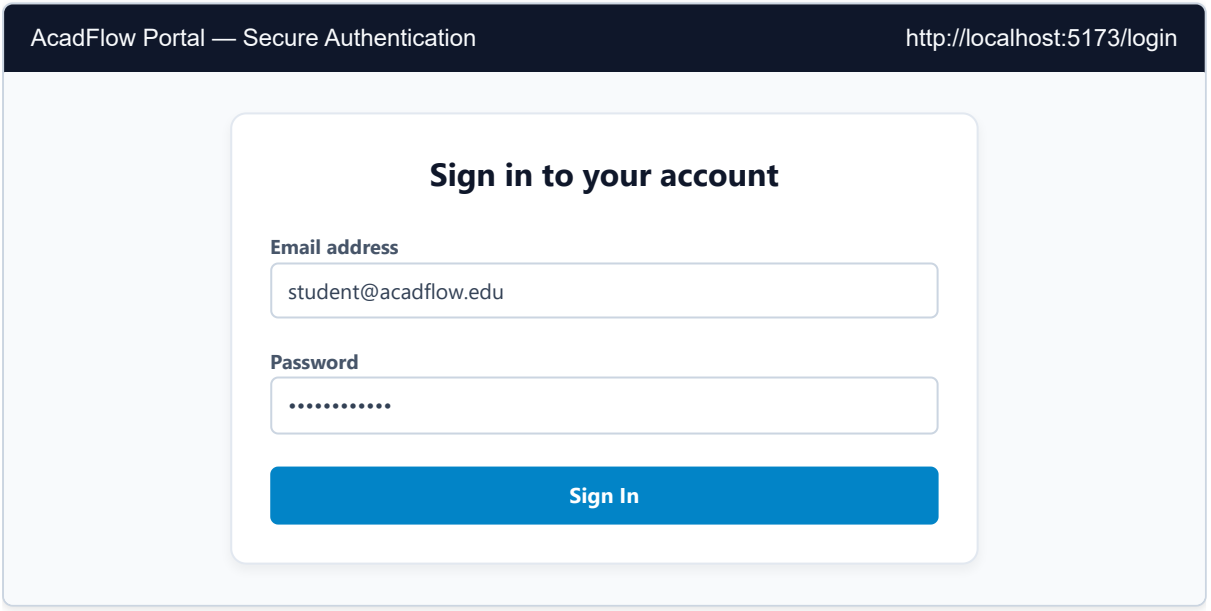


Figure 7.2: Secure Login & Authentication Interface

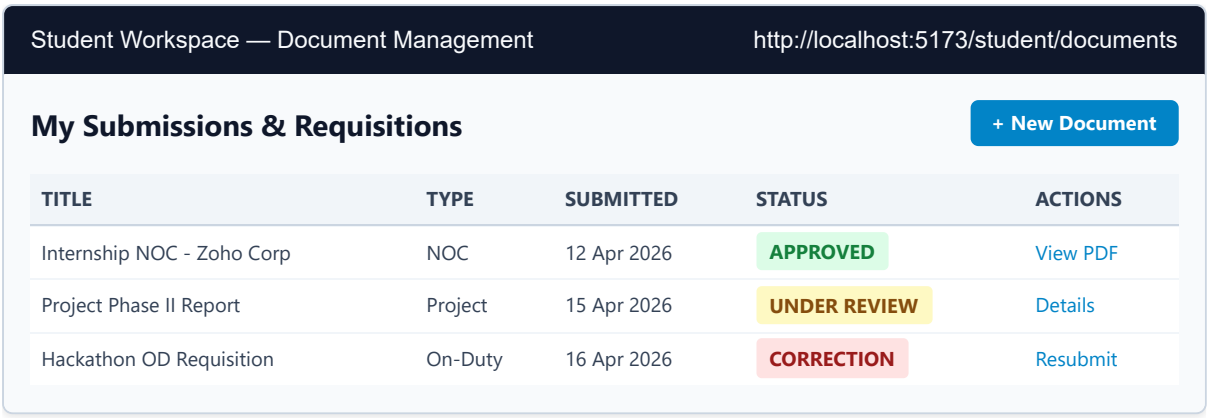


Figure 7.3: Student Dashboard & Document Submission Tracker

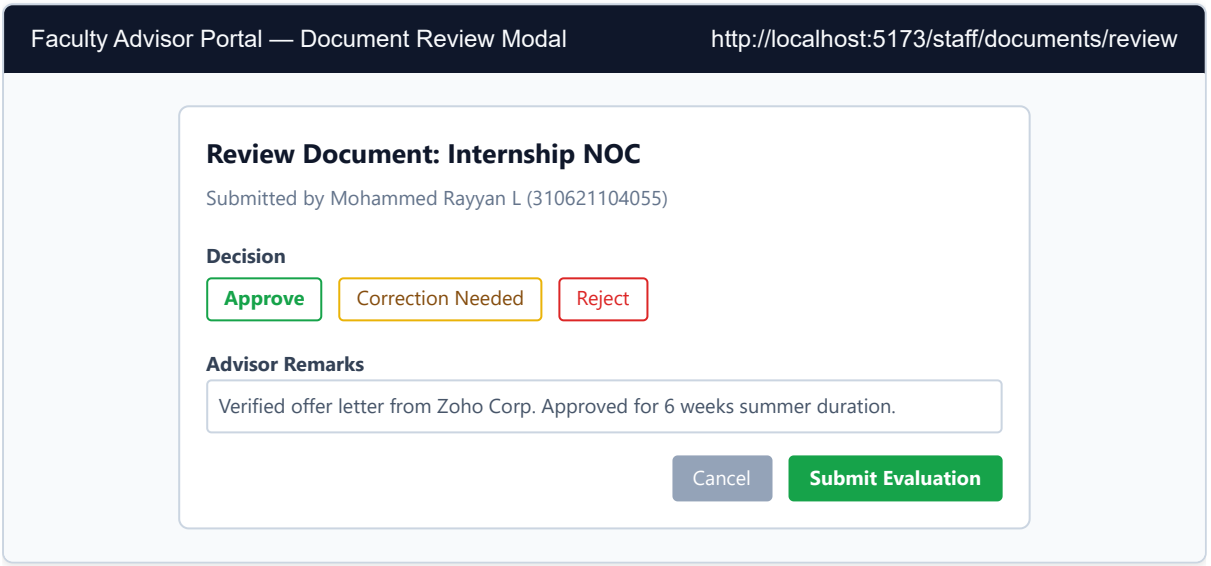


Figure 7.4: Staff Review, Remarks & Approval Modal

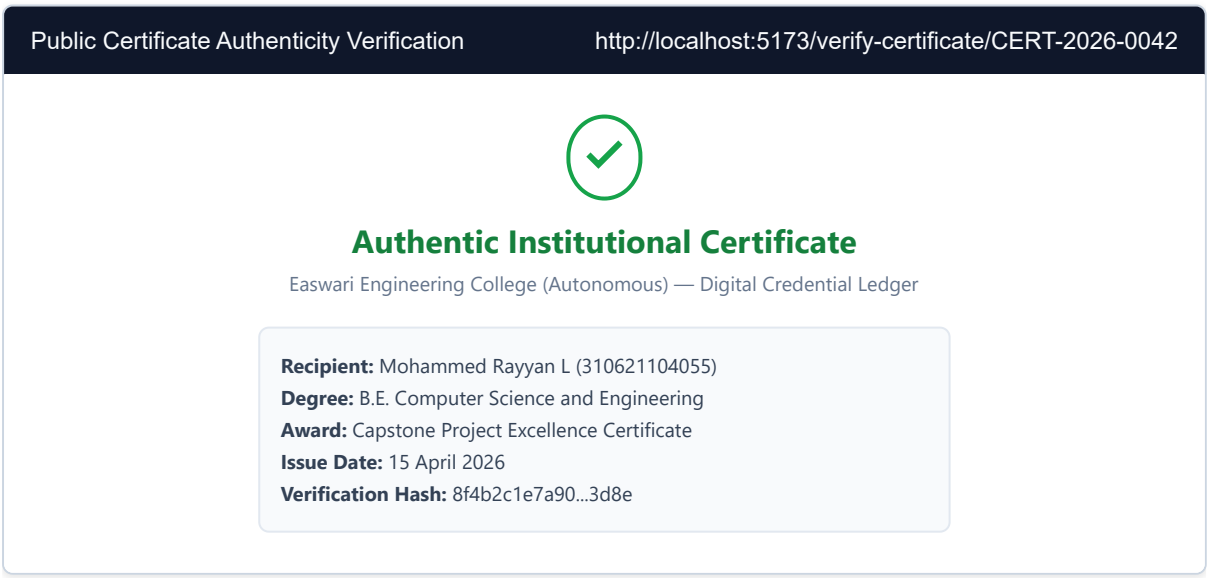


Figure 7.5: Digital Certificate with Public QR Code Verification Portal

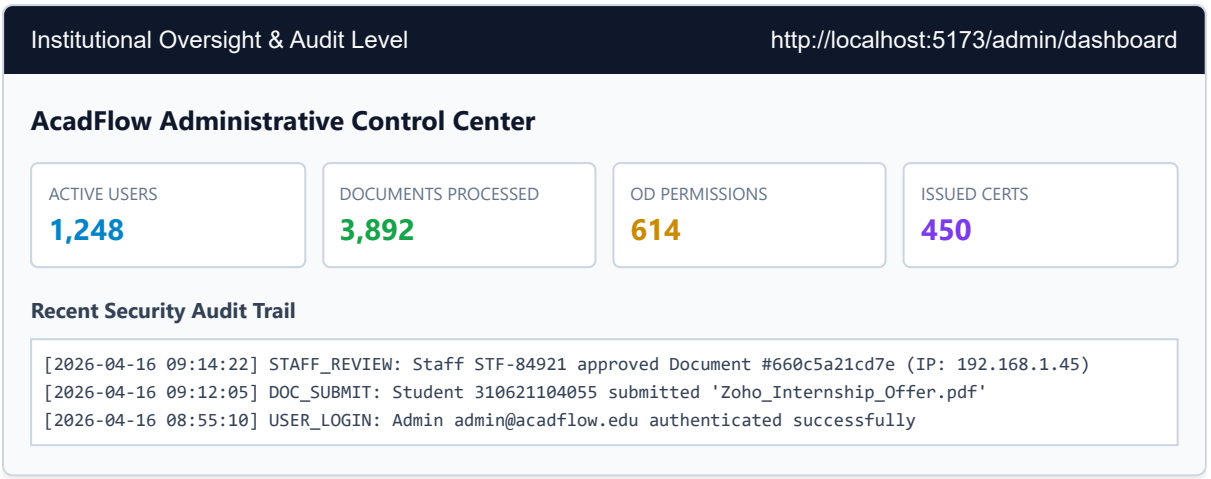


Figure 7.6: Admin Governance Dashboard & Real-time Audit Trail

REFERENCES

1. Kumar, A., Sharma, R., & Verma, P. (2021). "Architectural evaluation of enterprise academic document workflow systems in higher education," *IEEE Transactions on Learning Technologies*, vol. 14, no. 3, pp. 345–358.
2. Rodriguez, M., & Tanaka, K. (2022). "Securing digital academic credentials using cryptographic hash pointers and decentralized verification protocols," *ACM Inroads*, vol. 13, no. 2, pp. 52–63.
3. Chodorow, K. (2013). *MongoDB: The Definitive Guide: Powerful and Scalable Data Storage*, 2nd ed., Sebastopol, CA: O'Reilly Media.
4. Banks, A., & Porcello, E. (2020). *Learning React: Modern Patterns for Developing React Applications*, 2nd ed., Sebastopol, CA: O'Reilly Media.
5. Resig, J., Ferguson, R., & Bibeault, E. (2016). *Secrets of the JavaScript Ninja*, 2nd ed., Shelter Island, NY: Manning Publications.
6. Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*, Doctoral dissertation, University of California, Irvine.
7. Jones, M., Bradley, J., & Sakimura, N. (2015). "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF).
8. Stallings, W. (2017). *Cryptography and Network Security: Principles and Practice*, 7th ed., Boston, MA: Pearson.
9. Sommerville, I. (2015). *Software Engineering*, 10th ed., Boston, MA: Pearson.
10. Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach*, 9th ed., New York, NY: McGraw-Hill Education.
11. OWASP Foundation. (2021). "OWASP Top 10 Web Application Security Risks," Available online: https://owasp.org/www-project-top-ten/
12. Easwari Engineering College. (2024). *Academic Regulations and Project Documentation Norms*, Department of Computer Science and Engineering, Chennai, India.